深入浅出 Kafka

从一条消息到分布式事件流平台

目录

01 从业务事件到追加日志

02 动手观察 Topic、Offset 与消费组

03 集群架构：数据面与控制面

04 源码地图与阅读方法

05 Producer 源码：从 send 到批量发送

06 Broker 请求链路：网络、API 与延迟操作

07 存储引擎：Segment、Index 与页缓存

08 副本、ISR、高水位与故障恢复

09 KRaft：元数据日志、选举与快照

10 Consumer 源码：poll、Fetch 与提交

11 Consumer Group 与再均衡

12 幂等、事务与 Exactly-Once 边界

13 Retention、Compaction 与分层存储

14 协议、Record Batch 与兼容性

15 用 Kafka Connect 搬运数据

16 Kafka Streams：拓扑、状态与 Exactly-Once

17 安全模型：认证、授权与信任边界

18 生产部署与容量规划

19 性能优化：从基准测试到瓶颈定位

20 监控、运维与滚动变更

21 故障排查：从症状到证据链

22 源码精读实验与学习路线

23 术语、配置与参考书目

从业务事件到追加日志

先把 Kafka 看成一本持续追加的账本。每条事件写进去以后会保留一段时间，多个应用可以按自己的进度反复读取。

1.1 一条消息究竟是什么

假设订单系统刚刚完成一笔支付，它会产生“订单 42 已支付”这件事。在 Kafka 客户端中，这件事用一条 record 表示。record 通常包含 `topic`、可选的 `partition`、`timestamp`、`key`、`value` 和 headers。

Producer 会先用序列化器把 key 和 value 转成字节。Kafka 负责保存和传递这些字节，订单号、支付金额等业务含义由生产者和消费者共同约定。

Topic 用来给事件分类，例如 `orders`。一个 Topic 会分成若干 Partition，写入、复制和消费都以 Partition 为单位。每条记录写入分区后会得到一个 Offset，它表示这条记录在该分区中的位置。



图 1-1　一条业务事件的端到端路径。Kafka 保存字节与元数据，业务含义由应用负责。

1.2 为什么读完以后消息还在

一笔订单支付后，风控、结算和搜索索引都可能需要处理它。如果某个系统读完后消息立刻消失，其他系统就无法继续使用这份数据。

Kafka 分开管理数据和读取进度。Broker 根据 retention 或 compaction 策略决定数据保留多久；每个消费组用 committed offset 记录自己读到了哪里。这样，多个消费组可以独立读取同一份事件。应用修复以后，还可以把 offset 调回较早的位置重新处理。

日志的视角

数据保留多久，由 Topic 策略和磁盘容量决定。

订阅者的视角

读到哪里，由消费组的 offset 决定。

1.3 顺序保证的边界

Kafka 按分区维护顺序。订单 42 的“创建”“支付”“发货”需要依次处理时，可以把订单号作为稳定的 key。分区数和分区算法保持不变时，相同 key 会进入同一个分区。

多个分区各有自己的 Offset 序列，因此整个 Topic 没有统一的先后编号。单分区 Topic 可以提供全局顺序，同时把写入和同组消费的并行度限制为一路。



图 1-2　每个分区都有自己的 Offset 序列。跨分区事件需要依靠业务时间或其他字段判断先后。

1.4 吞吐、并行与成本

增加分区可以让更多 Broker 同时接收数据，也可以让一个消费组安排更多消费者并行处理。代价是集群需要维护更多日志文件、索引、副本和元数据，故障后的选主与迁移工作也会增加。

规划分区数时，先用真实消息大小、压缩方式和确认级别测出单分区吞吐，再结合峰值流量、消费并行度和恢复时间估算。分区数需要同时满足性能和运维成本两方面的要求。

源码锚点

- `clients/.../producer/ProducerRecord.java`：一条待发送记录的公共模型。
- `clients/.../common/TopicPartition.java`：Topic 与 Partition 的组合标识。
- `clients/.../consumer/OffsetAndMetadata.java`：消费组提交的位置与可选元数据。

第 02 章 · 动手实验

动手观察 Topic、Offset 与消费组

这一章先启动一个本地集群，再亲眼观察 Topic、Partition、Offset 和 Consumer Group 如何工作。

> 下面使用源码仓库中的脚本和开发配置。安装发行版时，请使用该版本自带的脚本、配置和 quickstart。

2.1 从源码构建并启动

```
./gradlew jar

KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
bin/kafka-storage.sh format --standalone \
  -t "$KAFKA_CLUSTER_ID" \
  -c config/server.properties

bin/kafka-server-start.sh config/server.properties
```

`format` 用来初始化空数据目录，并写入 cluster ID 和元数据。一个数据目录只需初始化一次。已有数据的目录再次使用新的 cluster ID 格式化后，会失去原来的集群身份。

2.2 建 Topic、写数据、读位置

```
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic orders --partitions 3 --replication-factor 1

bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
  --topic orders \
  --property parse.key=true --property key.separator=:

# 交互输入
order-1:created
order-2:created
order-1:paid

bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic orders --from-beginning \
  --formatter-property print.key=true \
  --formatter-property print.partition=true \
  --formatter-property print.offset=true
```

输出会显示三件事：相同 key 会稳定地落到同一分区，Offset 在各自分区内递增，再次使用 `--from-beginning` 时仍能读到先前写入的数据。

2.3 用消费组制造并观察 Lag

```
bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic orders --group order-demo

bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group order-demo
```

`CURRENT-OFFSET` 表示消费组已经提交的下一读取位置，`LOG-END-OFFSET` 表示分区日志末端。两者之差就是常说的 lag。它反映还差多少条记录；要判断业务是否超时，还要结合最老未处理事件的时间、生产速率和消费速率。

2.4 本地实验与生产环境的差别

本地做法	生产风险	生产思路
Broker 与 Controller 组合角色	控制面与数据面故障耦合	较大集群通常分离角色
单节点、RF=1	节点故障即不可用且无副本	常见关键数据 RF=3，跨故障域
PLAINTEXT	无加密、无身份、无授权	TLS/SASL/ACL 与网络边界
临时目录	容量、性能、生命周期不可控	独立磁盘、容量告警、保留策略

源码与文档锚点

- `docs/getting-started/quickstart.md`： 快速开始与命令示例。
- `config/server.properties`： 开发环境 KRaft 配置。
- `server-common/.../properties/MetaProperties.java`： 数据目录身份信息模型。

集群架构：数据面与控制面

一条订单事件由 Broker 保存和传递；Topic、分区和副本的安排由 KRaft Controller quorum 管理。分清这两条路径，架构会清楚很多。

3.1 数据路径、协调路径与控制路径

Producer 要写入 `orders-2` 时，会先查询这个分区当前由哪个 Broker 领导，然后直接向该 Broker 发送 Produce 请求。Controller 负责维护这份元数据，普通消息沿客户端到 Broker 的路径传输。

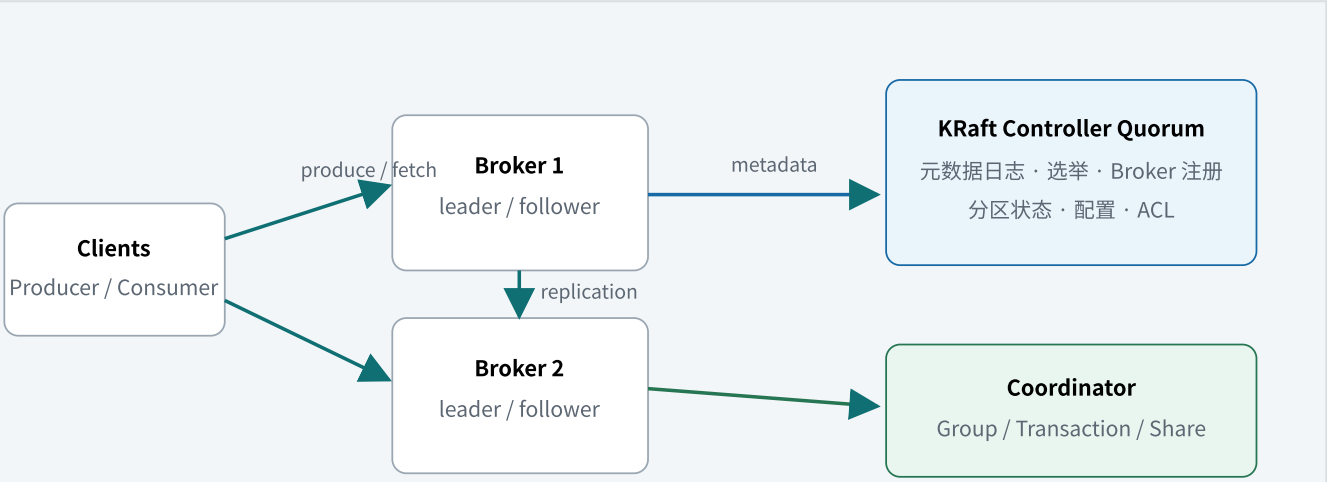


图 3-1　客户端数据请求不经过 Controller；Controller 管元数据，不承载普通消息流量。

- **数据路径：**Producer 和 Consumer 直接连接目标分区 leader，发送 Produce 或 Fetch。
- **协调路径：**Consumer 与 Group Coordinator 交互；事务 Producer 与 Transaction Coordinator 交互。
- **复制路径：**分区 follower 主动从 leader 拉取日志；leader 依据副本进度维护 ISR 与高水位。
- **控制路径：**Controller quorum 通过 Raft 复制元数据日志，active controller 计算状态变化并发布给 Broker。

3.2 为什么客户端只配置 bootstrap.servers

`bootstrap.servers` 只负责第一次联系集群。客户端从任意可用入口取得元数据后，会直接连接分区 leader 或 coordinator。

因此，bootstrap 地址要能连通，Broker 通过 `advertised.listeners` 公布的地址也要能从客户端网络访问。Docker、Kubernetes 和 NAT 环境中，常见现象是入口连接成功，随后连接元数据中的地址失败。

3.3 Controller 与 Coordinator 的职责

Controller 管理集群元数据，例如 Broker 注册、Topic、分区和副本状态。Group Coordinator 管理消费组成员、分区分配和 committed offset。Transaction Coordinator 管理事务状态以及事务涉及的分区。

这些组件可以运行在同一个 Broker 进程里，但各自维护独立的状态并处理不同协议。排障时先确认请求属于哪类职责，再查看对应日志和指标。

3.4 Controller 与 Coordinator 怎样被发现

Broker 通过配置中的 Controller 入口加入 KRaft quorum。动态 quorum 使用 `controller.quorum.bootstrap.servers`，`controller.listener.names` 指定 Controller 通信使用的 listener 名称。Broker 连接可用入口后，通过 KRaft 协议确认当

前 active controller；leader 切换后会沿同一机制更新连接目标。

```
controller.quorum.bootstrap.servers=controller-1:9093,controller-2:9093
controller.listener.names=CONTROLLER

# 静态 quorum 配置
controller.quorum.voters=1@controller-1:9093,2@controller-2:9093
```

Coordinator 运行在普通 Broker 上。客户端把业务 ID 交给 `FindCoordinator`：`group.id` 映射到 `__consumer_offsets` 的某个分区，`transactional.id` 映射到 `__transaction_state` 的某个分区。该内部 Topic 分区的 leader Broker 承担对应 Coordinator。

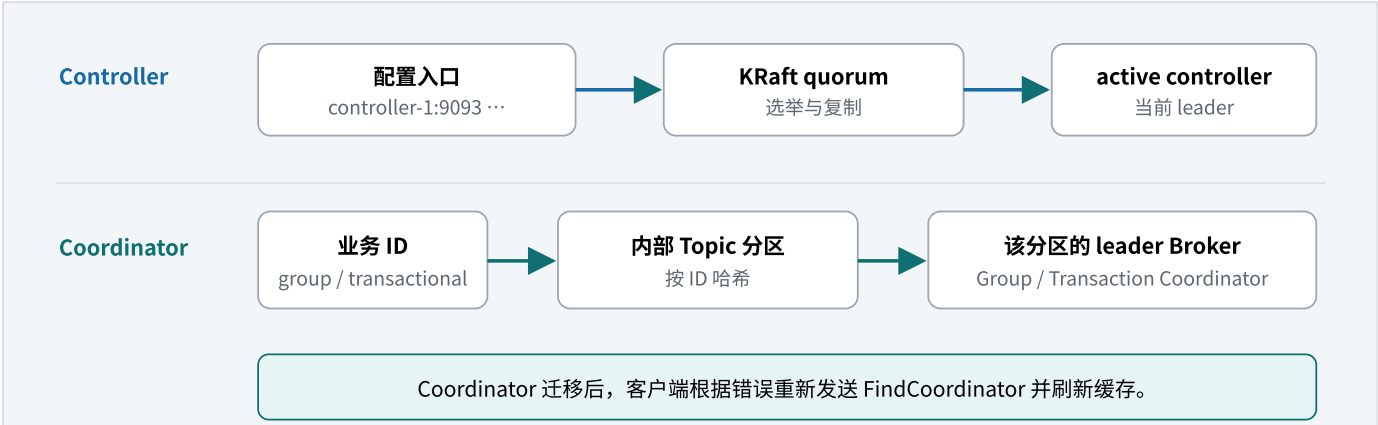


图 3-2　Controller 由 quorum 选举产生；Coordinator 的位置由内部 Topic 分区 leader 决定。

客户端	协调交互	消息数据交互
Consumer	Group Coordinator：入组、心跳、分配、位点	向分区 leader Fetch
事务 Producer	Transaction Coordinator：初始化、提交、中止	向分区 leader Produce
普通 Producer	查询并刷新 Topic 元数据	向分区 leader Produce

Kafka Streams 开启 exactly-once 处理后会同时使用两类 Coordinator：Group Coordinator 管理任务成员与输入分区，Transaction Coordinator 管理输出、状态 changelog 和输入位点的原子提交。

3.5 请求处理为何分层

Broker 收到请求后会经过几层处理。网络线程管理连接和字节收发，请求处理线程执行 Produce、Fetch 等协议逻辑，存储层负责日志读写。

这种分工让网络线程可以继续服务其他连接。请求队列、处理线程、磁盘或网络中的任一环节达到上限，最终都会表现为客户端延迟上升。

源码锚点

- `core/.../network/SocketServer.scala`： listener、acceptor、processor 与请求通道。
- `core/.../server/KafkaRequestHandler.scala`： 请求处理线程。
- `core/.../server/KafkaApis.scala`： 按 API key 分派协议请求。
- `raft/.../KafkaRaftClient.java`： KRaft 共识客户端。
- `GroupCoordinatorService.java: partitionFor` 与 `TransactionStateManager.scala: partitionFor`： 业务 ID 到内部 Topic 分区的映射。

第 04 章 · 源码导航

源码地图与阅读方法

阅读 Kafka 源码时，先跟完一条真实请求，再回头理解沿途遇到的模块。这样更容易把类和业务行为对应起来。

4.1 仓库模块怎么分

模块	核心职责	推荐入口
`clients`	Producer、Consumer、Admin、协议、序列化与网络客户端	`KafkaProducer`、`KafkaConsumer`、`NetworkClient`
`core`	Broker 请求入口、复制、配额及仍在迁移的 Scala 实现	`KafkaApis`、`ReplicaManager`
`storage`	日志段、索引、清理、远程存储	`UnifiedLog`、`LogSegment`、`LogCleaner`
`raft`	KRaft 共识、选举、日志复制与快照	`KafkaRaftClient`
`metadata`	Controller、元数据记录、image/delta	`QuorumController`、`MetadataImage`
`group-coordinator`	消费组协议、成员状态与 offset	`GroupCoordinatorService`、`GroupCoordinatorShard`
`transaction-coordinator` / `core`	事务协调与状态机	`TransactionCoordinator`
`connect` / `streams`	数据集成运行时与流处理库	Worker、Herder、KafkaStreams、Topology

4.2 一条 Produce 请求的阅读路线



4.3 用测试理解边界

生产代码展示正常流程，测试则把关键边界写得更集中。阅读 `UnifiedLogTest`，可以看到 Offset、日志截断、事务索引和恢复应满足哪些条件。阅读 `ProducerFailureHandlingTest`，可以看到生产者遇到不同错误时会重试、失败还是保持顺序。

```
# 单类测试示例
./gradlew clients:test --tests RequestResponseTest

# 存储模块定向测试
./gradlew storage:test \
  --tests org.apache.kafka.storage.internals.log.UnifiedLogTest
```

4.4 识别生成代码与公共 API

协议消息类由 JSON schema 生成。修改协议时，应编辑 resources 目录中的 message schema，再运行 `processMessages` 或 `processTestMessages` 重新生成代码。

公共接口、线协议、配置和指标都带有兼容性承诺，这类改动通常需要 KIP。完成标准除了编译和测试，还包括版本兼容、升级路径和文档。

> **推荐笔记格式：**入口方法 → 关键状态 → 成功条件 → 可重试错误 → 不可重试错误 → 指标/日志 → 对应测试。用这一模板读完一条链路，比孤立摘抄代码更容易形成可复用知识。

Producer 源码：从 send 到批量发送

调用 `send` 以后，记录会先进入客户端缓冲区，再由后台线程成批发送。这个过程同时决定了吞吐、延迟和失败处理方式。

5.1 send 进入异步批量流水线

订单服务调用 `KafkaProducer.send` 时，应用线程会先取得 Topic 元数据，把 key 和 value 序列化，再确定目标分区。准备好的记录随后被追加到 `RecordAccumulator`，`send` 返回一个 Future。

后台 `Sender` 线程持续检查哪些分区已经形成可发送的批次。它按目标 Broker 取出这些批次，构造 ProduceRequest，再交给 `NetworkClient` 发出。Broker 响应到达后，Producer 才会完成 Future 并调用回调。



图 5-1　应用线程主要做准备与入缓冲；后台线程承担批量网络发送。

5.2 分区选择：有 key 与无 key

记录显式指定了 partition 时，Producer 直接使用该分区。应用配置了自定义 Partitioner 时，分区选择交给自定义逻辑。其余情况由内置逻辑完成。

带 key 的记录通常根据序列化后的 key 字节计算分区，因此相同订单号会稳定地进入同一分区。无 key 的记录会在一段时间内集中到同一分区，积累出更完整的批次后再切换。这种黏性选择主要用于提高批量效率。

> **扩分区会改变 key 的去向：**分区数参与哈希计算。增加分区后，同一个 key 的新记录可能进入另一个分区。依赖 key 历史顺序的业务需要安排迁移期，或者使用显式路由。

5.3 批处理与背压

Producer 为每个分区分别积累批次。`batch.size` 控制批次的目标上限，`linger.ms` 允许低流量时多等一小会儿，以便把更多记录放进同一批。高流量会很快填满批次，此时无需等满 linger 时间。

所有未发送数据共同占用 `buffer.memory`。缓冲区耗尽后，调用 `send` 的线程最多等待 `max.block.ms` 获取空间。这里的等待就是 Producer 对上游施加的背压信号。

压缩同样以 batch 为单位。`gzip`、`snappy`、`lz4` 和 `zstd` 在压缩率、CPU 与延迟上各有取舍。选型时应使用真实消息和批次大小压测，同时观察 Producer CPU、Broker CPU、网络字节和端到端延迟。

5.4 acks、重试与最终时限

配置	真正控制的边界	常见误解
`acks=0`	发送后不等待 Broker 响应	应用无法从响应确认 Broker 是否保存成功
`acks=1`	leader 本地追加后确认	已经复制到 follower
`acks=all`	满足当前 ISR 与 min ISR 规则	等待所有 assigned replicas
`delivery.timeout.ms`	记录从 send 到最终成功/失败的总时限	只看 retries 就能推导最坏耗时

5.5 幂等生产者的边界

网络可能在 Broker 已经写入数据后中断。Producer 收不到响应，只能重试，这就可能造成重复。幂等生产者会为每个批次带上 producer ID、epoch 和分区序列号，Broker 据此识别同一生产会话中的重复重试，并维护分区内顺序。

这项能力覆盖 Producer 到 Kafka 的写入。数据库更新、HTTP 调用和业务重新投递仍要使用稳定 event ID、幂等表或其他协调方案处理。

源码锚点

- `KafkaProducer.java: send / doSend`：发送入口、序列化、元数据与追加。
- `RecordAccumulator.java: append / ready / drain`：分区队列与批次。
- `Sender.java: runOnce / sendProducerData`：请求形成、重试与响应。
- `NetworkClient.java: doSend / poll`：连接、版本协商、在途请求和响应。

Broker 请求链路：网络、API 与延迟操作

Produce 请求进入 Broker 后，会依次经过网络线程、请求队列、处理线程和存储层。延迟发生在哪一层，决定了排障方向。

6.1 从 socket 到 KafkaApis



Acceptor 接受新连接，Processor 负责读取网络帧。请求头解析完成后，请求进入 `RequestChannel`。RequestHandler 工作线程从队列中取出请求，再调用 `KafkaApis.handle`。

`KafkaApis` 根据 API key 分派请求。Produce 和 Fetch 通常继续进入 ReplicaManager；JoinGroup、OffsetCommit 等协调请求会进入对应的 Coordinator。

6.2 Produce 为什么可能延迟响应

采用 `acks=all` 时，leader 完成本地追加后还要等待足够多的同步副本跟上。这个等待可能持续数毫秒，也可能一直到请求超时。

Kafka 把等待中的请求放进 purgatory。副本条件满足、出现错误或到达超时时间时，请求才会完成。RequestHandler 可以继续处理其他请求，无需占着线程等待。

DelayedProduce

等待分区达到所需副本确认条件。

DelayedFetch

等待足够数据、超时或日志状态变化。

6.3 Fetch 为什么是长轮询

消费者发出的 Fetch 请求带有期望的最小数据量和最长等待时间。数据已经足够时，Broker 立即返回；数据较少时，请求会暂时等待，直到积累到目标大小或到达超时。

调大 `fetch.min.bytes` 往往能减少请求次数并提高吞吐。低流量场景需要等待更多数据，因此单条记录的延迟也会增加。

6.4 延迟拆解

一次写入的端到端延迟，可以拆成客户端成批等待、网络往返、Broker 请求排队、本地追加、副本确认和响应回写。逐段观察这些时间，才能判断瓶颈在客户端、Broker 线程、磁盘、网络还是副本。

现象	优先怀疑	证据
RequestQueueTime 上升	handler 线程不足或被慢请求占满	RequestHandlerAvgIdlePercent、CPU、请求分布
LocalTime 上升	存储、配额、序列化或内部处理	磁盘延迟、页缓存、GC、请求类型
RemoteTime 上升	副本/远程读取等等待	ISR、复制延迟、远程存储指标
ResponseQueueTime 上升	网络线程回写能力	NetworkProcessorAvgIdlePercent、带宽

6.5 配额如何保护共享集群

共享集群中，一个批量任务可能突然提高写入量，挤占其他应用的网络和请求线程。Kafka 可以按 user、client ID 或两者组合设置配额，并在客户端超过额度时返回 throttle time，让客户端主动放慢请求。

`producer_byte_rate` 和 `consumer_byte_rate` 控制网络字节，`request_percentage` 控制请求处理时间占比，`controller_mutation_rate` 限制创建 Topic、改配置等控制面操作。配额值应来自容量预算和租户 SLO，并通过 throttle 指标验证效果。

源码锚点

- `KafkaApis.scala: handle / handleProduceRequest / handleFetchRequest`。
- `ReplicaManager.scala: appendRecords / fetchMessages`。
- `server/.../purgatory` 与 `core/.../server/Delayed*.scala`。
- `server/.../quota/ClientQuotaManager.java`： 客户端配额计算与节流。

存储引擎：Segment、Index 与页缓存

一个分区在磁盘上是一组顺序排列的日志段。Kafka 通过批量追加、稀疏索引和页缓存，让大量磁盘数据仍能高效读写。

7.1 一个分区就是一个逻辑日志

分区目录由多个 segment 组成，当前活跃段负责接收新记录。当它达到大小、时间或索引限制时，Kafka 会执行 roll，创建一个新的活跃段。已经关闭的旧段随后可以参与删除或日志压缩。

每个 segment 用 base offset 命名。一个段通常包含日志数据文件、Offset 索引、时间索引以及事务索引。日志保存完整记录，索引用较小的空间帮助快速定位。



图 7-1　索引按间隔采样记录位置；稀疏结构降低空间与维护成本。

7.2 为什么顺序写仍然很快

Producer 先把多条记录组成 batch，通常还会按 batch 压缩。Broker 收到后按顺序追加到日志，Consumer 和 follower 也倾向于连续读取一段数据。这样的访问方式能充分利用磁盘顺序吞吐和批量网络 I/O。

热数据主要由操作系统页缓存承接。因此，Broker 的 JVM 堆用于对象和运行状态，磁盘日志的主要缓存空间留给操作系统管理。

7.3 Offset 查询如何落到文件

1. 根据 segment base offset 找到可能包含目标 offset 的段。
2. offset index 用“相对 offset → 文件位置”定位到靠近目标的位置。
3. 从该位置顺序解析 record batch，找到满足条件的记录。

按时间查找时，Kafka 会先通过 time index 找到接近目标时间的位置，再顺序扫描并确认记录。日志文件保存事实数据，索引负责加速查询；索引损坏后可以根据日志重建。

7.4 mmap：像访问内存一样查索引

假设 Consumer 要从 offset 1,008,420 开始读取。Kafka 先选中 base offset 为 1,000,000 的 segment，再把目标换算成相对 offset 8,420。接下来，它在 offset index 中查找最接近且小于等于 8,420 的索引项，得到日志文件位置，然后从那里顺序扫描 record batch。

Kafka 用 `MappedByteBuffer` 映射索引文件。映射建立后，Java 代码可以直接读取一段虚拟地址；这些地址对应操作系统 Page Cache 中的文件页。首次访问某一页时，内核按需装入数据并补齐页表。后续查询命中同一批缓存页时，索引查找主要发生在内存中。

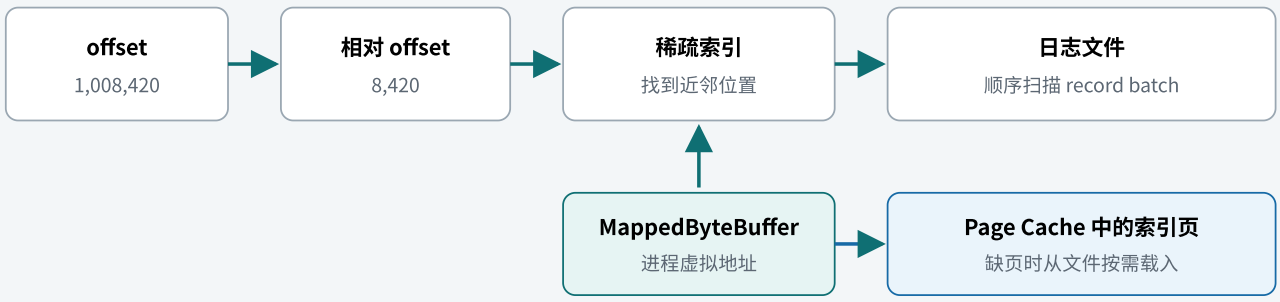


图 7-2　mmap 优化的是索引访问：虚拟地址映射到文件缓存页，查询结果再引导日志顺序读取。

索引文件体积较小、查询频繁且访问位置分散，这类负载很适合 mmap。日志数据文件更大，Kafka 读取到目标批次后还要把记录发送给 Consumer，发送阶段会走另一条优化路径。

7.5 sendfile：让日志直接进入网络路径

Broker 返回 Fetch 响应时，记录已经按 Kafka 批次格式保存在日志段中。明文连接可以把选中的文件区间交给 `FileChannel.transferTo`。在支持相应能力的操作系统和 JDK 上，内核会直接把 Page Cache 中的文件页送入 socket 路径，Kafka 进程省去两次大块字节搬运：Page Cache 到用户缓冲区，以及用户缓冲区到 socket 缓冲区。

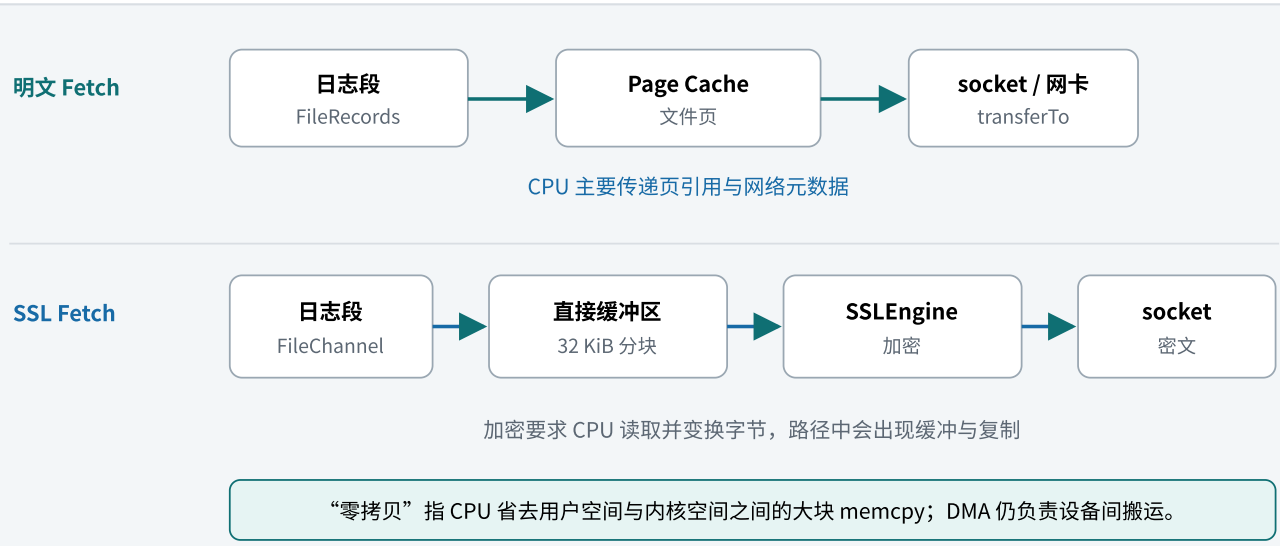


图 7-3　明文连接有机会使用内核零拷贝；SSL 路径需要让字节经过加密引擎。

`mmap` 负责“查” · `transferTo` 负责“发” · `SSLEngine` 负责“加密”

7.6 Flush、fsync 与可靠性

Broker 追加的数据通常先进入操作系统缓存，再由后台机制刷入磁盘。Kafka 还会把同一分区复制到其他 Broker。采用 `acks=all` 并设置合适的 min ISR 后，客户端成功意味着多个同步副本已经接收这批数据。

每条记录都同步 fsync 会显著降低吞吐。生产设计应把磁盘可靠性、副本数量、确认条件和故障域放在一起评估。

源码锚点　`UnifiedLog`、`LogSegment`、`LogManager` 串起日志管理；`AbstractIndex.createMappedBuffer` 建立索引映射；`FileRecords.writeTo` 把文件区间交给网络通道；`PlaintextTransportLayer.transferFrom` 调用 `FileChannel.transferTo`；`SslTransportLayer.transferFrom` 分块读取并加密。

副本、ISR、高水位与故障恢复

客户端看到“写入成功”时，究竟有几个副本已经保存数据？Leader、ISR 和高水位共同回答这个问题。

8.1 Leader、Follower 与 ISR

每个分区有一个 leader 和若干 follower。客户端通常向 leader 读写，follower 持续从 leader 拉取数据。Kafka 把当前追赶及时的副本放入 ISR，也就是同步副本集合。

ISR 会影响 `acks=all` 的确认条件，也提供安全的 leader 候选。assigned replicas 表示规划中的副本，alive replicas 表示当前存活的副本，ISR 进一步表示其中同步状态满足要求的副本。

8.2 LEO、HW 与 LSO



图 8-1　LEO 是各副本日志末端；HW 划出可稳定读取的边界。事务下还要考虑 LSO。

LEO 是某个副本的日志末端，也就是下一条记录将要写入的位置。**HW** 划出普通消费者可以稳定读取的范围，它会随着 ISR 的复制进度向前推进。

事务场景还会使用 **LSO**。未完成事务会让 LSO 停在更早的位置；`read_committed` 消费者只显示已提交事务中的数据，并以 LSO 作为当前读取上界。

8.3 RF=3、min ISR=2、acks=all

RF=3 表示每个分区保存三份副本，min ISR=2 要求写入确认时至少有两个同步副本。一个副本落后时，剩余两个 ISR 仍可满足 `acks=all`；同步副本只剩一个时，Broker 会拒绝这类写入。

这是一组常见起点。最终配置要根据业务允许的数据损失、恢复时间、故障域和成本进行验证。

> `NotEnoughReplicas` **是可靠性告警：**它表示同步副本数量已经低于写入承诺。处理重点是恢复副本、磁盘或网络。临时降低 min ISR 会缩小数据安全余量，需要明确授权、时间窗和恢复条件。

8.4 Leader 故障时发生什么

下面用 `orders-0` 的一次故障串起完整过程。该分区有三个副本，Broker 1 是 leader，Broker 2 和 Broker 3 是 follower：

```
leader epoch = 7
ISR = [Broker 1, Broker 2, Broker 3]
HW  = 100

Broker 1: leader    LEO = 103，保存 offset 0..102
Broker 2: follower  LEO = 102，保存 offset 0..101
Broker 3: follower  LEO = 100，保存 offset 0..99
```

Offset 0..99 位于 HW 之前，构成稳定前缀。100..102 仍处在复制推进过程中，各副本拥有的尾部长度不同。

第一步：Controller 判断 Broker 1 已经失效

Broker 会持续向 active controller 发送心跳。心跳租约到期、Broker 主动关闭、日志目录离线等事件，都会让 Controller 更新该 Broker 或副本的可用状态。Controller 随后检查受影响分区，并从 ISR 中移除失效副本。

故障检测需要一个时间窗口。窗口过短会把瞬时网络抖动放大成频繁选主，窗口过长则会增加分区恢复时间。客户端在这段时间里可能看到连接断开、请求超时或 leader 暂时不可用。

第二步：Controller 选择新 Leader

正常选主会寻找仍然存活且位于 ISR 的副本。当前实现还可以在 ISR 为空时使用符合条件的 ELR。Controller 依据已提交的分区元数据判断候选资格，选主过程侧重安全集合与副本顺序，而非临时向所有 Broker 查询 LEO 再比较最大值。

在这个例子中，Broker 2 成为新 leader。Controller 生成分区变更记录，将 leader 改为 Broker 2，从 ISR 移除 Broker 1，并把 leader epoch 从 7 推进到 8。该记录写入 KRaft 元数据日志，经多数 Controller 确认后成为新的权威状态。

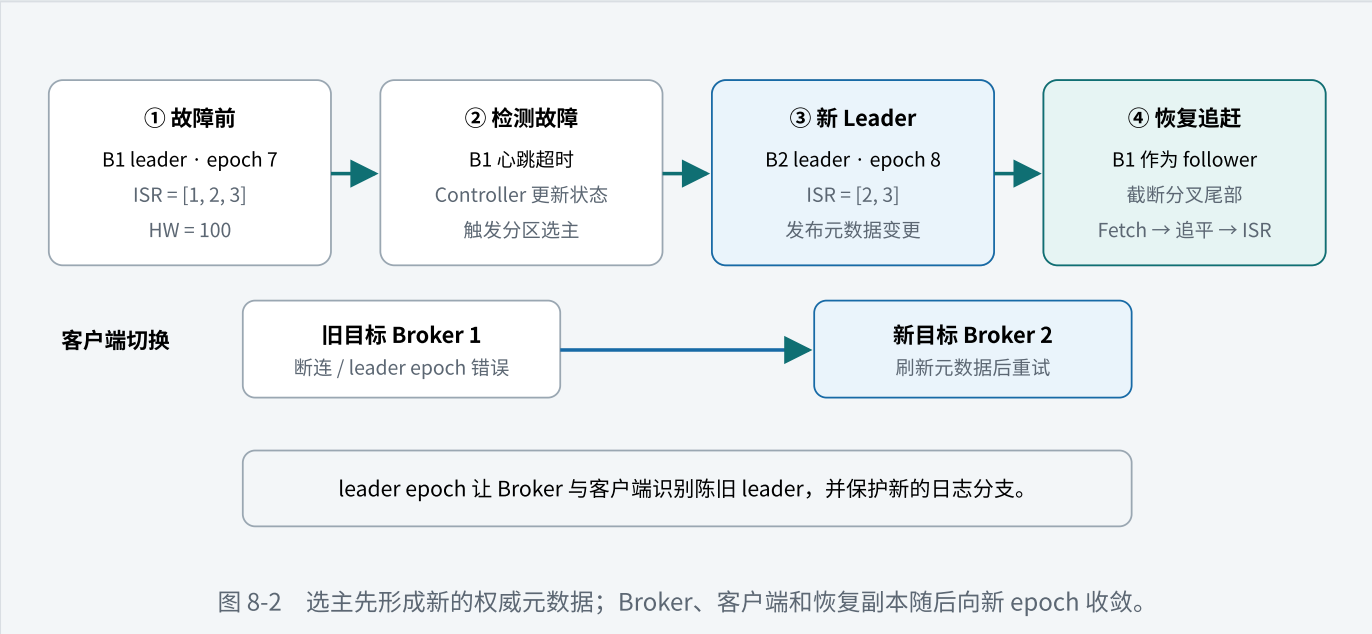


图 8-2　选主先形成新的权威元数据；Broker、客户端和恢复副本随后向新 epoch 收敛。

第三步：各 Broker 应用新的元数据

Broker 持续读取 KRaft 元数据更新。Broker 2 收到分区 delta 后执行 leader 转换，停止该分区的 follower fetcher，建立 leader 侧复制状态并开始处理 Produce、Fetch。Broker 3 保持 follower 身份，把复制目标切换到 Broker 2。

新 leader 刚接管时会从安全的 HW 边界继续维护可见性。随着 follower 报告新的抓取位置，HW 再次向前推进。分区在选主完成后恢复服务，ISR 和 HW 的完全稳定通常还需要一段追赶时间。

第四步：客户端刷新元数据并重试

仍向 Broker 1 发送请求的客户端会遇到断连、`NOT_LEADER_OR_FOLLOWER`、`FENCED_LEADER_EPOCH` 等结果。Producer、Consumer 和 follower 随后刷新 Topic 元数据，找到 leader epoch 8 的 Broker 2，并从各自位置继续请求。

Producer 的在途请求可能进入重试。幂等 Producer 使用 producer ID、epoch 和 sequence 识别重复批次；Consumer 根据当前 fetch position 继续读取。应用仍要按 event ID 处理端到端重试带来的业务重复。

第五步：旧 Leader 恢复、截断并追赶

Broker 1 恢复后从元数据中得知自己已经是 follower。此时 Broker 2 可能已经在 epoch 8 写入 offset 102..105，而 Broker 1 的 offset 102 仍属于旧 epoch 7，两条日志从 offset 102 开始分叉。

副本	故障前	新 Leader 写入后	恢复完成
Broker 1	epoch 7，LEO 103	离线；offset 102 是旧分支	截断到 102，再拉取 epoch 8 数据
Broker 2	epoch 7，LEO 102	epoch 8 leader，LEO 106	继续担任 leader
Broker 3	epoch 7，LEO 100	从 Broker 2 追赶	追平并维持 ISR 身份

Follower 会携带自己最后看到的 leader epoch 发起复制请求。新 leader 返回相应 epoch 的结束 Offset 或在 Fetch 响应中指出分叉位置。Broker 1 据此找到共同历史边界 102，删除从 offset 102 开始的旧尾部，再从 Broker 2 拉取新的 102..105。

Broker 1 追到 leader 的有效日志并满足副本滞后条件后，leader 才会申请把它重新加入 ISR。Controller 将新的 ISR 写入元数据日志，整个恢复过程才算收敛。

哪些消息可以保留下来

正常 ISR 选主会保留 HW 之前的稳定前缀。HW 之后的记录取决于新 leader 是否已经拥有它们：共同拥有的尾部可以继续保留，只有旧 leader 保存的尾部会在恢复时被截断。

写入确认方式	Leader 故障时的含义
`acks=all`，且 min ISR 配置合理	成功响应建立在足够同步副本已接收记录的基础上，适合承诺故障后的数据安全。
`acks=1`	旧 leader 本地追加后即可响应；尚未复制到新 leader 的已响应记录可能位于被截断尾部。
请求结果未知	Broker 可能已经追加，但响应在网络中丢失；客户端重试与业务幂等共同处理这种不确定性。

8.5 Unclean leader election 的代价

开启 unclean leader election 后，Controller 可以在 ISR 之外选择新 leader。这样有机会更快恢复服务，但新 leader 可能缺少旧 leader 已经接收的数据，客户端会看到数据丢失或日志位置回退。

这项配置表达的是“在特定故障下优先恢复可用性，并接受数据风险”。启用条件应由业务恢复目标和数据承诺共同决定。

源码锚点

- `ReplicationControlManager.java` 与 `PartitionChangeBuilder.java: electLeader`：副本状态、候选资格与选主记录。
- `BrokerMetadataPublisher.scala`、`ReplicaManager.scala: applyDelta` 与 `Partition.scala: makeLeader / makeFollower`：Broker 应用新元数据。
- `AbstractFetcherThread.scala: maybeTruncate / truncationOffset`：根据 leader epoch 确定截断点并恢复复制。
- `UnifiedLog.java`：HW、日志起始/末端与读取隔离相关状态。

KRaft：元数据日志、选举与快照

创建 Topic、Broker 上下线和分区选主都会改变集群元数据。KRaft 把这些变化写进一条由 Controller quorum 共同复制的日志。

9.1 Controller quorum 保存什么

Topic、Partition、Broker 注册、配置、配额和 ACL 的每次变更，都会编码成元数据记录并追加到 KRaft 日志。日志按顺序保存集群发生过的变化。

Controller 和 Broker 使用 image 表示某个 Offset 上的完整元数据视图，使用 delta 表示两次视图之间的增量。加载快照并重放后续记录，就能恢复当前状态。

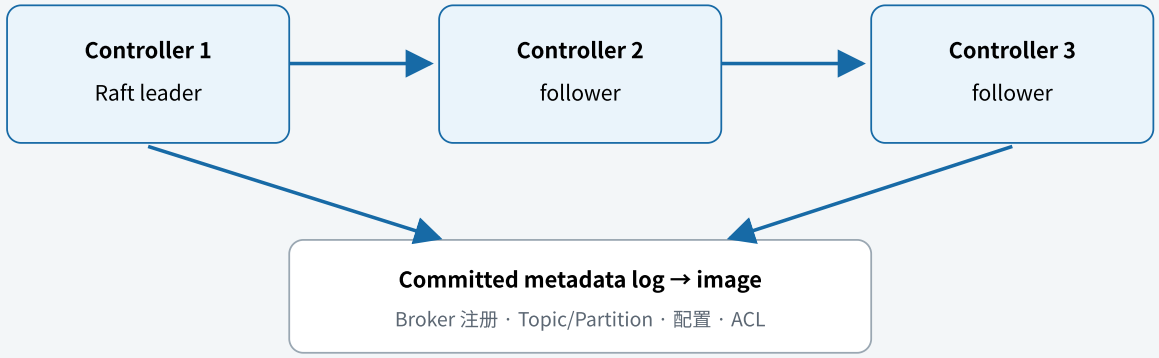


图 9-1　三节点 quorum 可容忍一个节点故障；多数派确认即可提交。

9.2 2N+1 的真正含义

Raft 需要多数派才能提交记录。3 个 voter 组成的 quorum 可以容忍 1 个节点故障，5 个 voter 可以容忍 2 个。增加节点会提高可容忍的故障数，也会让每次提交等待更多节点。

Controller voter 适合放在彼此网络延迟较低、故障域相互独立的位置。跨地域网络延迟会直接增加创建 Topic、选主和配置变更等控制面操作的耗时。

9.3 Epoch、选举与日志一致性

Controller 需要新 leader 时，候选者会发起选举。获得多数票的候选者进入新的 epoch，并成为 leader。投票规则会比较候选者的日志进度，让拥有足够新日志的节点获得领导权。

Leader 把新记录复制给 follower，多数派确认后再推进 high watermark。quorum 失去多数派时，已有元数据仍可供 Broker 使用，新的元数据变更会暂停提交。

9.4 快照为什么必要

元数据日志会持续增长。每次启动都从第一条记录开始重放，恢复时间也会随之增长。快照保存某个已提交 Offset 上的完整状态，节点启动时先加载快照，再重放后续增量即可。

快照与对应的日志位置、cluster ID 和节点身份共同构成恢复上下文。迁移或恢复时需要使用目标版本提供的工具和流程，保持这些信息一致。

9.5 观察 quorum

```
bin/kafka-metadata-quorum.sh --bootstrap-server localhost:9092 \
  describe --status

bin/kafka-metadata-quorum.sh --bootstrap-controller localhost:9093 \
  describe --replication
```

输出中先确认 leader 和 epoch，再查看 high watermark、各 voter 的 lag 与最后抓取时间。多数派保持连通并持续复制，quorum 才能提交新的元数据记录。

源码锚点

- `raft/.../KafkaRaftClient.java`：poll、选举、复制与状态转换。
- `metadata/.../QuorumController.java`：控制器事件与元数据变更。
- `metadata/.../image` 与 `metadata/.../delta`：状态视图和增量。
- `metadata/src/main/resources/common/metadata`：元数据记录 schema。

Consumer 源码：poll、Fetch 与提交

消费者已经从 Broker 拉到哪里、已经交给业务哪里、故障后准备从哪里恢复，这是三个不同的位置。理解它们以后，重复与漏处理问题会清晰很多。

10.1 poll 同时推进多件事

应用调用 `poll(Duration)` 时，Consumer 会确认订阅和消费组状态，推动网络请求与协调事件，从 fetch buffer 收集记录，然后更新当前 position。一次 poll 同时推进网络、组协调和记录交付。

当前源码包含 Classic 与 Async 两套内部实现。公共 `KafkaConsumer` API 通过 `ConsumerDelegate` 连接到具体实现，因此阅读源码时先确认实际使用的是哪条路径。



10.2 Fetch 直接发往分区 leader

Consumer 完成入组并取得 assignment 后，会根据 Topic 元数据连接各分区的 leader Broker。Group Coordinator 继续处理心跳、组状态和位点提交；消息批次沿 Fetch 路径直接返回 Consumer。

协调交互

Consumer → Group Coordinator：Heartbeat、Join/Sync 或 ConsumerGroupHeartbeat、OffsetCommit。

数据交互

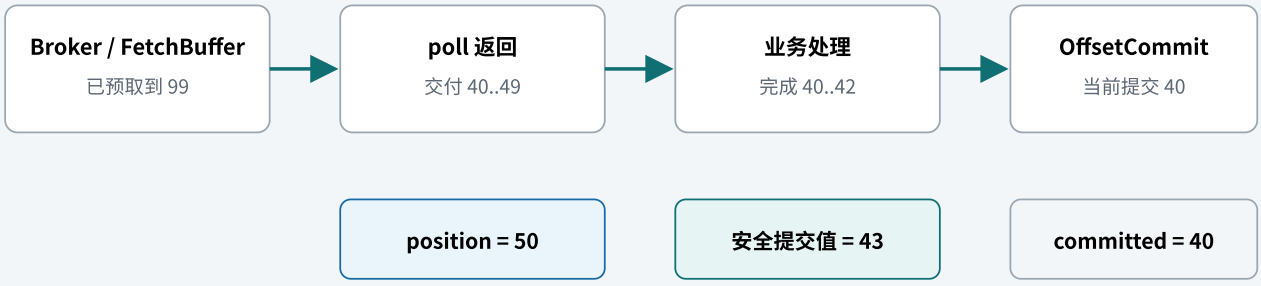
Consumer → 分区 leader：Fetch orders-0、orders-1，并持续推进各分区位置。

假设 `orders-0` 的 leader 在 Broker 1，`orders-1` 的 leader 在 Broker 2，而 Group Coordinator 在 Broker 3。一次 `poll()` 可以推动到 Broker 1 和 Broker 2 的 Fetch，同时维护与 Broker 3 的协调状态。三个角色恰好落在同一 Broker 时，协议职责依然各自独立。

10.3 position 与 committed offset

阶段	含义	由谁维护
预取缓冲	Broker 响应已经到达客户端，记录仍在 FetchBuffer 中	Consumer 内部
`position()`	下一条准备交给应用的记录；poll 返回记录时自动推进	Consumer 进程内状态
处理完成位置	业务逻辑已经成功处理到哪里	应用按分区自行维护
Committed offset	消费组持久化的恢复起点	Group Coordinator / `__consumer_offsets`

公共 API 中，`position(tp)` 和 committed offset 是最重要的两个位置。预取缓冲可以包含更靠后的记录，但 position 只在记录通过 `poll()` 交给应用时推进。Kafka 负责记录“已经交付到哪里”，业务是否处理成功则由应用判断。



进程崩溃后，本地缓冲与 position 消失；新实例从 committed offset 40 恢复。

图 10-1 “已经拉到”“已经交给应用”“已经处理”和“已经提交”可以处在四个不同位置。

Committed offset 保存的是“下一条准备恢复的记录”。处理完 offset 41 后应提交 42；处理完 40..42 后应提交 43。提交值采用下一个 Offset，可以让恢复直接从尚未完成的记录开始。

以上图为例，Consumer 崩溃后会从 committed offset 40 恢复，因此 40..49 可能再次交给应用，这形成 at-least-once。应用按照 event ID 幂等处理即可安全吸收重复。

如果应用在处理前提交 50，随后只完成到 42 就发生崩溃，新实例会从 50 恢复，43..49 会被越过。异步并行处理时，应按分区提交“连续完成前缀的下一个 Offset”；后面的任务即使先完成，也要等待前面的空洞补齐。

10.4 自动提交的处理边界

`enable.auto.commit=true` 会在 poll 推动的时间点周期提交位置。Consumer 能看到记录何时交给应用，却看不到数据库更新或远程调用何时完成。

如果应用把 records 交给异步线程后立即继续 poll，自动提交可能越过仍在处理的任务。并行消费需要按分区追踪连续完成的位置，并在分区被撤销前停止接收新任务、等待已有任务收敛，再提交安全位置。

10.5 max.poll.interval.ms 与 max.poll.records

`max.poll.records` 限制一次 poll 交给应用的记录数。Consumer 已经抓取到本地的数据可以超过这个数量。
`max.poll.interval.ms` 规定两次 poll 之间允许经过的最长时间。

当单批处理时间接近这个上限时，可以减小每批记录数，把慢外部调用移出 poll 线程，或者增加分区级并行。处理时延分布比单纯调大超时更能说明容量是否充足。

10.6 reset 只在没有有效 offset 时生效

`auto.offset.reset` 可以设为 `earliest`、`latest`，也可以使用 `by_duration:PT1H` 这类 ISO-8601 时长。它在两种情况下决定起点：分区还没有 committed offset，或者原有 Offset 已经因数据保留而越界。

消费组已有有效提交时，`earliest` 会保持当前位置。回放历史数据需要显式重置 group offset。执行前先停止消费者，并确认 Topic、分区和目标时间范围。

源码锚点

- `KafkaConsumer.java` 与 `ConsumerDelegate.java`：公共 API 到内部实现。
- `AsyncKafkaConsumer.java: poll`、`ClassicKafkaConsumer.java: poll`。
- `FetchRequestManager.java`、`FetchBuffer.java`、`FetchCollector.java`。
- `SubscriptionState.java`：订阅、分配、position 与 pause 状态。

Consumer Group 与再均衡

当一个风控消费者追不上订单流量时，可以启动更多同组实例。Kafka 会把分区分给这些实例，让它们并行处理。

11.1 并行度上限来自分区

同一个消费组内，一个分区在同一时刻只交给一个成员。6 个分区最多提供 6 路有效并行；第 7 个成员没有可分配的分区，会处于空闲状态。另一个消费组拥有独立分配，可以从头到尾再读一遍同一 Topic。

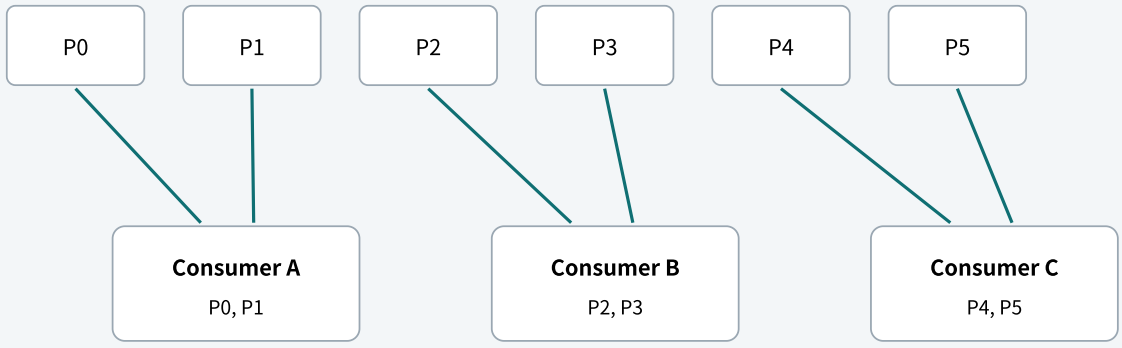


图 11-1　6 个分区最多提供 6 路同组消费并行；更多成员会空闲。

11.2 Group Coordinator 保存什么

Group Coordinator 记录组内有哪些成员、它们订阅了什么、分区如何分配，以及组当前的 generation 或 epoch。Committed offsets 也由协调器负责管理，并持久化到 Kafka 内部日志。

消费者先找到本组的 Coordinator，再通过加入、同步或新组协议的心跳过程取得分区。承载协调器的 Broker 故障后，对应状态可以从内部日志恢复到其他 Broker。

11.3 多个 Consumer 同时启动时怎样分配

使用相同 `group.id` 的 Consumer 会映射到同一个 `__consumer_offsets` 分区，因此会找到同一个 Group Coordinator。经典协议中，第一个成员加入空组后，Coordinator 可以等待 `group.initial.rebalance.delay.ms`，把紧接着启动的其他成员收进同一轮初始分配。

经典协议会选择一名成员作为 group leader。Coordinator 把成员订阅交给它，group leader 运行选定的 assignor，再通过 `SyncGroup` 提交结果。新版 consumer group protocol 由 Broker 端 assignor 计算分配，成员通过 `ConsumerGroupHeartbeat` 逐步取得 assignment。

同组有效并行度 ≤ Topic 分区数；额外成员保持心跳，等待分区空缺。

11.4 Consumer 怎样得知分区被收回

成员加入、离开或超时，Topic 分区数变化，以及 Coordinator 迁移，都可能引起分区重新分配。分配变化期间，部分消费会短暂停顿，尚未提交的记录也可能被新成员再次处理。

以经典协议为例，Consumer B 加入后，Coordinator 把组切换到准备再均衡状态。Consumer A 的下一次 Heartbeat 收到 `REBALANCE_IN_PROGRESS`，客户端据此标记需要重新入组。通知随着 A 发出的请求返回，Coordinator 保持服务端角色。



A 在 `poll()` 推进协调流程时应用新 assignment。它停止为已撤销的 `orders-2` 创建 Fetch，只向应用交付当前仍拥有的分区数据。旧 generation 的 Heartbeat 和位点提交会被 Coordinator 拒绝；新版协议使用 member epoch 提供同类隔离。

A 长时间没有心跳时，Coordinator 会在 `session.timeout.ms` 到期后移除该成员，并把它的分区分给其他成员。A 恢复后携带旧 generation 或 member epoch 发出请求，会先被拒绝，再重新执行入组流程。

调整前 · generation 1

Consumer A：orders-0、orders-1、orders-2。

调整后 · generation 2

Consumer A：orders-0、orders-1；Consumer B：orders-2。

Eager 策略会先撤销所有分区，再完成整组分配。Cooperative 策略只迁移需要变化的分区，通常能缩小停顿范围。应用需要正确处理多轮回调，以及分区所有权逐步变化的过程。

11.5 静态成员资格

为实例设置稳定的 `group.instance.id` 后，短暂重启可以继续使用原来的成员身份，从而减少分区来回迁移。每个并发实例都要使用唯一 ID；两个实例使用同一 ID 时，旧实例会被 fencing。

在容器平台上，可以把该 ID 绑定到稳定的实例编号，并在实例整个生命周期内保持一致。

11.6 RebalanceListener 的正确用途

- `onPartitionsRevoked`：停止相关工作，等待分区内已接收任务完成，提交连续完成位置。
- `onPartitionsAssigned`：初始化分区状态，必要时 seek 到外部保存的位置。
- `onPartitionsLost`：所有权已经丢失，此时停止提交这些分区的位置，也停止继续产生业务副作用。

源码锚点

- `group-coordinator/.../GroupCoordinatorService.java`：协议入口与分片路由。
- `GroupCoordinatorShard.java`：组与 offset 状态机。
- `GroupMetadataManager.java`、`OffsetMetadataManager.java`。
- `AbstractCoordinator.java`、`ConsumerCoordinator.java`：经典协议的入组、心跳和重新入组。
- `SubscriptionState.java` 与 membership/heartbeat manager：本地 assignment、epoch 与新版协议。

幂等、事务与 Exactly-Once 边界

订单处理程序常常要“读一条、写结果、提交进度”。Kafka 事务可以让这三个动作一起提交，或者一起中止。

12.1 三种能力各自解决什么问题

能力	提供的保证	能力边界
业务幂等	同一 event ID 重做不产生额外副作用	多条 Kafka 写入的原子性
幂等生产	同一生产会话网络重试不在分区内重复追加	跨分区原子提交、外部数据库
Kafka 事务	多分区写入和消费 offset 原子提交/中止	任意外部系统的分布式事务

12.2 consume-transform-produce 的事务流程



配置 `transactional.id` 后，Producer 先通过 `FindCoordinator` 找到 Transaction Coordinator，并在 `initTransactions()` 中取得 producer ID 与 epoch。`beginTransaction()` 建立客户端事务上下文，随后的消息仍然直接写入各目标分区 leader。

Transaction Coordinator 记录事务当前处于开始、准备提交、已提交还是已中止状态，也记录事务涉及哪些分区。`sendOffsetsToTransaction` 还会让 Group Coordinator 接收事务型位点提交，把输入进度写入 `__consumer_offsets`。

`read_committed` 消费者会跳过已中止事务的数据，并且读取位置受 LSO 限制。处理失败后，应用中止事务，输入 Offset 保持在原处，下一次可以重新处理。

12.3 一个事务可以跨 Topic 和分区

一次事务可以写入多条消息、多个 Topic 和多个分区，也可以包含消费位点。例如支付服务同时写入 `payments-0`、`inventory-2` 和 `audit-1`，这三个分区的 leader 可以位于不同 Broker。

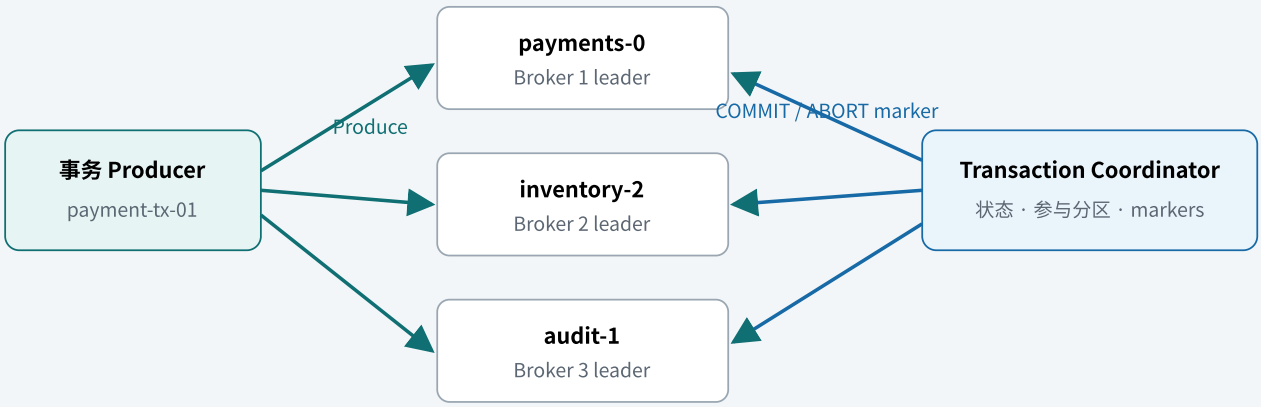


图 12-1　消息由 Producer 直接写入分区 leader；Coordinator 统一决定提交或中止结果。

各分区会在不同时间完成追加。提交时，Coordinator 先记录准备提交状态，再向所有参与分区写入 `COMMIT` marker，最后记录事务完成；中止时写入 `ABORT` marker。`read_committed` 消费者据此统一显示已提交消息，并跳过已中止消息。

事务只包含一条输出消息时，仍可以把“输出消息”和“输入位点”放进同一提交边界。这正是 consume-transform-produce 场景中 Transaction Coordinator 的核心价值。

12.4 transactional.id 与 fencing

`transactional.id` 为生产者提供一个可以跨重启延续的事务身份。新实例初始化后会取得更高的 epoch，Coordinator 随后拒绝旧实例继续使用这个身份写入，这个过程称为 fencing。

因此，`transactional.id` 要稳定地绑定到应用实例或任务。同一时间运行的两个实例需要使用不同的 ID。

12.5 写数据库怎么办

Kafka 事务覆盖 Kafka 内的记录和 Offset。业务还要写普通数据库时，通常采用以下方案：

- **Transactional outbox：**业务数据与 outbox 行在同一数据库事务提交，再由 CDC/relay 写 Kafka。
- **Inbox/幂等表：**以 event ID 去重，并把“已处理标记”与业务更新放进同一数据库事务。
- **可补偿工作流：**对跨系统长事务记录步骤与补偿动作，接受最终一致。

> **实用判断：**如果输入和输出都在 Kafka 内，事务或 Streams EOS 很合适；一旦跨出 Kafka，先定义外部系统的幂等键、提交点、重试窗口和对账方式。

源码锚点

- `clients/.../producer/internals/TransactionManager.java`：producer ID、epoch 与事务状态。
- `core/.../coordinator/transaction/TransactionCoordinator.scala`。
- `TransactionMarkerChannelManager.scala` 与 `KafkaApis.handleWriteTxnMarkersRequest`：向参与分区写入完成标记。
- `storage/.../ProducerStateManager.java`：序列号、事务与快照状态。
- `storage/.../TransactionIndex.java`：中止事务索引。

Retention、Compaction 与分层存储

Kafka 按 Topic 策略清理数据。Delete 控制历史保留时长，Compaction 保留每个 key 的最新状态，消费进度与清理时机彼此独立。

13.1 消费进度与日志清理彼此独立

Consumer 读取消息时只推进本消费组的 committed offset。日志清理由 Topic 的 `cleanup.policy` 触发，因此一个消费组读完消息后，其他消费组仍可从各自的位置读取同一批数据。

策略	保留规则	典型用途
`delete`	超过 `retention.ms` 或分区大小上限后，按日志段清理	事件历史、审计流、业务消息
`compact`	逐步保留每个 key 的最新值与必要 tombstone	状态 changelog、配置快照
`compact,delete`	同时应用 key 压缩与时间/大小上限	有限窗口内的最新状态

慢消费组需要在保留窗口内追上写入速度。假设该组提交在 offset 420，而日志最早可读位置已经推进到 900，下一次 Fetch 会得到 Offset 越界错误。Consumer 随后根据 `auto.offset.reset` 选择新起点，或者由运维人员显式重置位置。

> **保留时间是数据可回放窗口：**容量规划要覆盖业务积压、故障恢复和离线回放的最长时间，并为 segment roll 与清理检查周期留出余量。

13.2 cleanup.policy=delete

Broker 以 segment 为单位检查保留时间和日志总大小。旧 segment 达到 `retention.ms` 或 `retention.bytes` 的条件后，会先被标记，再经过删除延迟从磁盘移除。

活跃 segment 会继续接收记录，直到发生 roll。即使其中最早的记录已经过期，Kafka 也会等它成为关闭段后再整体清理。因此实际保留时间还会受到 segment 大小、roll 时间和检查间隔影响。

13.3 cleanup.policy=compact

以用户资料 Topic 为例，同一个用户可能先后写入多个版本。Log Cleaner 扫描已经关闭的 dirty segments，为 key 建立去重映射，再重写日志，逐步清除旧版本。清理在后台分段进行，因此新值写入后，旧值还会保留一段时间。

value 为 null 的记录称为 tombstone，用来表达“删除这个 key”。Tombstone 也会保留一段时间，让下游和离线副本有机会观察到删除事件。

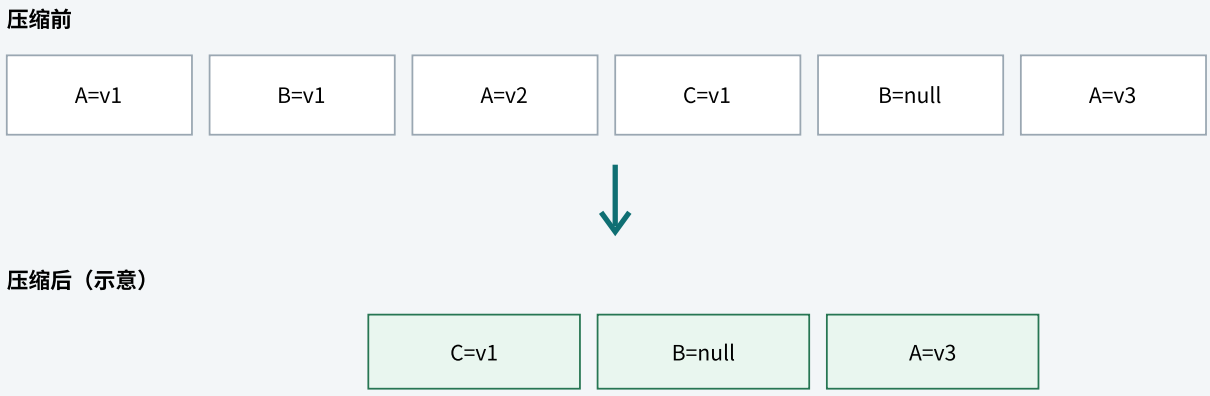


图 13-1　压缩后保留 key 的最新状态。被清理的记录会留下 Offset 空洞，现有 Offset 保持原值。

13.4 delete 与 compact 可以组合

`cleanup.policy=compact,delete` 会同时保留每个 key 的最新状态，并为日志设置时间或大小上限。完整 changelog 需要支持从头重建状态，过早删除会让重建缺少数据。只需要有限时间窗口内最新状态的 Topic，则可以用组合策略控制存储成本。

13.5 分层存储改变了什么

分层存储会把较老的关闭段复制到远程存储，只在 Broker 本地保留近期热数据。这样可以延长逻辑保留时间，并减少本地磁盘占用。

远程层同时带来上传、索引、缓存和对象存储费用。回放旧数据时还会增加远程读取延迟。容量规划需要同时计算本地盘和远程存储，监控也要覆盖上传失败与远程 Fetch。

13.6 容量粗算

```
本地原始容量 ≈ 峰值写入字节/秒 × 本地保留秒 × 副本因子 ÷ 压缩比 × 安全系数
```

公式之外，还要预留 segment 与索引、分区重分配的临时副本、日志清理的重写空间，以及流量增长余量。压缩比应使用真实 record batch 和生产配置测量，普通文本压缩工具无法反映批次结构与消息分布。

源码锚点

- `storage/.../LogCleaner.java` 与 `Cleaner.java`。
- `storage/.../LogManager.java`：删除、flush 与检查点调度。
- `clients/.../FetchCollector.java`：处理 Offset 越界与 reset。
- `storage/.../remote`：Remote Log Manager 与远程元数据。

协议、Record Batch 与兼容性

客户端和 Broker 往往分批升级。Kafka 用 API 版本、协议 schema 和 feature level 约定双方都能理解的能力。

14.1 请求如何被版本化

每个请求头都带有 API key、API version 和 correlation ID。客户端先发送 ApiVersions 请求，了解 Broker 支持哪些版本，再为 Produce、Fetch 等 API 选择双方兼容的版本。

协议 schema 会标明字段在哪些版本出现，flexible versions 还提供可扩展的 tagged fields。旧客户端继续使用自己理解的旧版本，新客户端则可以在新 Broker 上启用更多字段和能力。

14.2 为什么 Producer 发送的是 batch

同一批记录共享许多属性，例如 base offset、时间戳类型、压缩方式、producer ID、epoch 和 base sequence。Record Batch 把这些属性集中放在批次头中，避免每条记录重复携带。

压缩也以 batch 为单位。批次中相似数据越多，通常压缩效果越好，网络请求次数也越少。Broker 复制时可以直接传递整个 batch，无需理解业务 value。

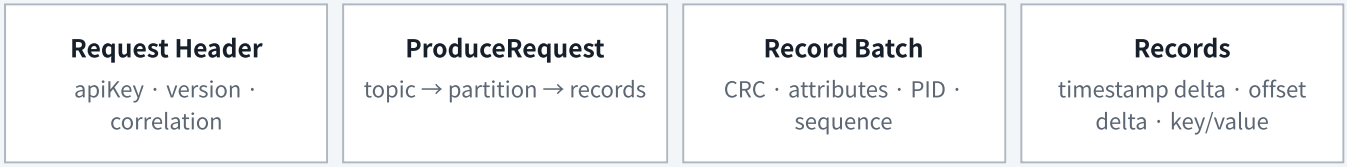


图 14-1　协议层层嵌套；幂等与事务元数据位于 record batch，而非业务 value。

14.3 metadata.version 与软件版本各自控制什么

软件版本决定节点运行哪套代码，协议版本决定一次请求使用哪种格式，`metadata.version` 则控制集群启用哪些元数据能力。它们分别处在不同层次。

滚动升级通常先替换所有节点的二进制，同时保持原有 feature level。集群稳定后，再按目标版本文档提升 feature。某些 feature 会改变持久化格式或行为，升级方案需要提前写清可回退的阶段和最终边界。

14.4 Schema Registry 解决的是业务 schema

Kafka 协议把 key 和 value 当作字节。订单字段如何新增、删除，默认值如何设置，以及 reader/writer schema 如何配合，属于业务数据治理。

使用 Avro、Protobuf 或 JSON Schema 时，团队可以通过 Schema Registry 管理版本和兼容规则。Kafka 客户端与 Broker 协议兼容，只说明传输链路可以工作；业务记录能否正确解析仍由数据 schema 决定。

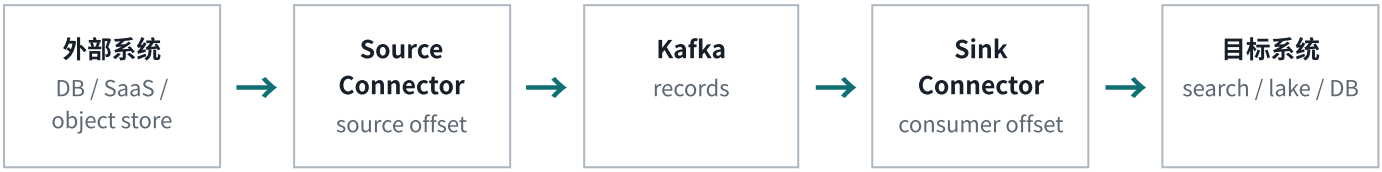
源码锚点

- `clients/src/main/resources/common/message`：请求/响应 schema。
- `clients/.../common/protocol/ApiKeys.java`：API 与版本范围。
- `clients/.../common/record`：Record Batch 编解码。
- `server-common/.../Feature.java` 与 metadata feature 代码。

用 Kafka Connect 搬运数据

数据库、对象存储和搜索系统之间经常需要持续搬运数据。Kafka Connect 为这类任务提供统一的配置、进度记录、分配和重启机制。

15.1 Source 与 Sink



Connector 读取配置并把工作拆成若干 Task，Task 才是实际执行数据搬运的单元。Source Task 从外部系统读取数据，并用 source offset 记录外部位置；Sink Task 以消费组方式读取 Kafka，再写入目标系统。

`tasks.max` 给出并行 Task 的上限。实际并行度还取决于源系统能切成多少份、Topic 有多少分区，以及目标系统能承受多少并发写入。

15.2 Distributed mode 的内部 Topic

Distributed mode 使用内部 Topic 保存 connector 配置、source offset 和运行状态。Worker 重启或任务迁移后，会从这些 Topic 恢复控制面状态。

这些 Topic 需要合适的副本因子、压缩策略和访问权限。删除它们会造成配置、进度或状态丢失，应把它们纳入 Connect 集群的备份与监控范围。

15.3 转换与错误处理

Converter 在 Kafka 字节和 Connect Data 之间转换。SMT 适合对单条记录做改名、过滤、字段增删等轻量变化。复杂 join、聚合和大量外部调用更适合放在流处理应用或独立服务中。

错误容忍配置可以把坏记录写入 DLQ。DLQ 还需要配套告警、责任人、修复流程和重放工具，才能真正形成可恢复的错误处理闭环。

15.4 Connect 的安全边界

插件、Converter、SMT、ConfigProvider 和 REST extension 都运行在 Worker JVM 内，并继承进程权限。因此，安装插件相当于允许这段代码读取 Worker 可访问的数据并执行本地操作。

REST API 可以读取配置、停止任务或删除 connector，属于集群级管理入口。互相隔离的团队适合使用独立 Connect 集群，并分别控制插件目录、凭据和管理网络。

> **生产基线：**只安装可信插件；限制 `plugin.path`；REST API 仅暴露在管理网络或认证代理之后；严格控制 connector client config override；机密通过受控 provider 注入。

源码与安全模型锚点

- `connect/runtime/.../Worker.java`：Task/Connector 生命周期。
- `connect/runtime/.../distributed/DistributedHerder.java`：分布式协调。
- `docs/security/security-model-connect.md`：REST、插件和信任边界。

Kafka Streams：拓扑、状态与 Exactly-Once

Kafka Streams 把一连串业务处理写成拓扑。输入分区带来并行度，本地 state store 保存计算状态，changelog 负责故障恢复。

16.1 DSL 与 Processor API

假设订单流需要过滤测试订单、按门店聚合金额，再与门店资料关联。DSL 可以用 KStream、KTable 和 GlobalKTable 组合 filter、map、aggregate、window 与 join，代码直接对应业务步骤。

Processor API 提供更细的处理器、状态存储和时间控制。大多数拓扑可以先用 DSL 表达；需要自定义状态机、特殊调度或底层记录控制时，再使用 Processor API。

16.2 KStream 与 KTable 的语义差异

KStream：事件事实

每条记录都是一次独立事件，例如每次支付尝试。

KTable：键值更新

同 key 新值替代旧状态，例如用户当前等级。

每次支付尝试都是独立事实，适合用 KStream 表示。用户当前等级会被新值更新，适合用 KTable 表示。压缩 Topic 常用来恢复表状态，而拓扑中的 KStream 或 KTable 类型决定运行时如何解释记录。

16.3 有状态处理与恢复



图 16-1　本地状态提升处理性能，changelog 让任务能在其他实例恢复。

按新的 key 聚合或 join 时，数据需要先按这个 key 重新分区，Streams 会视情况创建 repartition topic。有状态操作则会把 state store 的变更写入 changelog topic。

任务迁移到另一实例后，可以从 changelog 恢复本地状态。这些内部 Topic 直接影响结果正确性和恢复时间，副本、保留和权限需要按生产数据同等管理。

`num.standby.replicas` 可以让其他实例提前维护 state store 的备用副本。故障切换时，备用副本只需追赶少量 changelog，恢复会更快；代价是额外的本地磁盘、网络读取和实例容量。

16.4 时间与窗口

流处理通常按事件发生时间分配窗口。记录可能因网络或上游重试而乱序到达，grace period 决定窗口关闭后还能接收多晚的数据。

Stream time 由应用已经观察到的事件时间向前推进，窗口关闭和最终结果都受它影响。较短 grace 能更早产出最终结果，也会丢弃更多合理迟到的数据；较长 grace 会增加状态保留和结果等待时间。

16.5 Streams 的 EOS

Exactly-once processing 使用 Kafka 事务，把输入 Offset、状态 changelog 和输出 Topic 一起提交。任务失败时，这些变化一起中止，恢复后可以从原来的输入位置重做。

Processor 同时调用外部支付接口或写普通数据库时，这些操作处在 Kafka 事务之外，还需要幂等键、outbox 或补偿流程。

16.6 测试拓扑与恢复过程

`TopologyTestDriver` 可以在单进程中输入测试记录、推进时间并读取输出，适合验证分支、聚合、窗口和状态存储。它让业务规则测试保持快速和确定。

生产前还需要带真实 Broker 的集成测试，覆盖序列化、内部 Topic、事务、任务迁移和重启恢复。对有状态拓扑而言，“新实例能否在目标时间内恢复状态”与计算结果同样重要。

运行时监控应展开到 thread、task 和 state store，重点观察处理速率、提交延迟、错误记录、状态恢复进度和再均衡。应用实例存活时，某个 task 仍可能已经停滞。

源码锚点

- `streams/.../KafkaStreams.java`：应用生命周期。
- `streams/.../processor/internals/StreamThread.java` 与 `StreamTask.java`。
- `streams/.../state`：状态存储与 changelog。
- `streams/.../Topology.java`、`StreamsBuilder.java`。
- `streams/test-utils/.../TopologyTestDriver.java`：拓扑单元测试工具。

第 17 章 · 安全

安全模型：认证、授权与信任边界

一条 Kafka 连接要经过链路加密、身份认证和资源授权。密钥管理与网络隔离把这些能力连接成完整的安全边界。

17.1 四层防线

TLS
加密链路并验证服务端；mTLS 也可认证客户端。

SASL
SCRAM、Kerberos、OAuthBearer 等身份机制。

ACL / Authorizer
决定 principal 可对哪些资源执行什么操作。

Listener 和网络策略用于分隔客户端、inter-broker 与 controller 流量。Controller 和 Broker 的内部端口只向受信服务器网络开放，普通客户端通过专用 listener 访问集群。

17.2 TLS 常见失败

- 证书 SAN 不包含客户端使用的主机名，导致 hostname verification 失败。
- 只部署服务端 keystore，却遗漏客户端 truststore 或中间 CA 链。
- 设置 `ssl.client.auth=requested` 后误以为未带证书的客户端会被拒绝。
- 证书轮换只替换部分节点，或没有同时支持旧/新信任链的过渡窗口。

17.3 SASL 与 ACL 的分工

SASL 用来确认客户端身份，Authorizer 决定这个身份可以操作哪些资源。消费者通常同时需要 Topic READ 和 Group READ；事务生产者还会使用 TransactionalId 权限。

授权设计从应用身份出发，列出它要访问的 Topic、Group、Cluster 和 TransactionalId，再授予完成职责所需的最小操作集合。

17.4 机密与插件

JAAS 密码、私钥和 OAuth client secret 应存放在受控的密钥系统中，通过运行时配置注入。镜像、源码、命令历史和教程示例都应保持无密钥。

ConfigProvider 和其他插件运行在进程内，可以读取 Worker 权限范围内的数据。选型和部署时需要把插件代码本身纳入信任审查。

> 安全问题分类应先确认攻击者所处的信任边界。已获管理员或本机权限后的行为属于既有高权限边界；远程攻击面则按 `SECURITY.md` 和 `docs/security/` 的模型评估。

文档与源码锚点

- `SECURITY.md` 与 `docs/security-model.md`。
- `docs/security/encryption-and-authentication-using-ssl.md`。
- `docs/security/authentication-using-sasl.md`。
- `server/.../authorizer` 与 `metadata/.../authorizer`。

生产部署与容量规划

生产集群从业务目标开始设计。流量、保留时间、故障容忍和处理时限确定以后，节点、分区、副本、磁盘和网络才有计算依据。

18.1 先写清四个输入

1. **流量：**平均/峰值 records 与 bytes，每条消息大小分布，读写放大。
2. **生命周期：**本地与远程保留时间、压缩比、是否 compaction。
3. **可靠性：**允许丢多少、允许停多久、同时容忍几个故障域失败。
4. **处理：**消费组数量、每组处理能力、最大可接受 lag 与回放时间。

18.2 Broker 与 Controller

开发环境可以让一个进程同时承担 Broker 和 Controller 角色。较大的生产集群通常分离两种角色，使控制面避开业务流量、数据盘抖动和 Broker 长 GC 的影响。

Controller 节点保存元数据日志，重点是稳定的低延迟网络、可靠磁盘和独立故障域。业务分区存放在 Broker 上，因此 Controller 无需配置与 Broker 相同规模的数据盘。

18.3 分区数的双向约束

```
所需分区数 ≥ max(峰值写入吞吐 ÷ 单分区写基准, 目标消费并行度)
```

这个公式给出一个起点。单分区吞吐要使用真实消息大小、压缩算法、`acks`、复制因子和生产硬件测量。

得到初始分区数后，还要验证 leader 分布、Broker 网络、故障恢复时间、文件句柄和控制面负担，并为未来增长预留空间。

18.4 磁盘与网络

Broker 的入站流量包括 Producer 写入和 follower 拉取，出站流量包括 Consumer 读取和副本复制。RF=3 意味着一份业务数据最终要写入三份副本，副本之间还会消耗网络带宽。

消费者大规模回放、分区重分配和 Broker 恢复会让后台流量远高于平时。容量测试需要覆盖这些非稳态场景，并为故障恢复保留磁盘与网络余量。

资源	需要预留的非稳态场景
磁盘容量	保留增长、segment 清理、compaction 重写、分区迁移临时副本
磁盘吞吐	副本追赶、批量回放、恢复索引、日志清理
网络	跨 Broker 复制、消费者回放、重分配、远程存储上传/读取
CPU	压缩/解压、TLS、协议处理、GC 与大量小请求

18.5 机架感知与故障域

配置 `broker.rack` 后，副本分配器会尽量把同一分区的副本放到不同机架或可用区。上线后仍需检查实际副本分布，以及跨区带宽和网络费用。

Controller voter、Broker 副本和客户端的部署位置，要共同满足同一个故障假设，例如“失去一个可用区后仍能写入和消费”。

18.6 Topic 治理

- [ ] 命名、owner、数据等级、schema、保留与预期吞吐有登记。
- [ ] 关键 Topic 明确 RF、min ISR、acks 与恢复目标。
- [ ] 限制无界自动建 Topic，避免拼写错误制造生产数据孤岛。
- [ ] 增加分区前评估 key 重映射、下游并行和不可逆性。
- [ ] 删除与缩短保留必须有影响分析和可验证回滚/恢复方案。

18.7 多集群与灾备

单集群副本用于承受集群内部的节点或故障域失效。跨区域灾备需要第二个集群，以及明确的数据复制和客户端切换方案。

MirrorMaker 2 等方案可以把选定 Topic、消费组 Offset 和部分配置同步到另一集群。复制通常异步进行，因此要明确可接受的数据缺口 RPO、恢复时间 RTO、复制方向、Topic 命名和循环防护。

灾备演练需要覆盖目标数据、DNS 或客户端配置切换、ACL 与凭据、Schema、Connect/Streams 状态、消费 Offset 映射、回切和重复处理。每个环节都要有可观察的成功条件。

性能优化：从基准测试到瓶颈定位

Kafka 的吞吐、延迟、可靠性和资源成本相互牵制。有效调优从业务目标和可重复基准开始，再沿 Producer、Broker、Consumer 的数据路径寻找瓶颈。

19.1 先把性能目标写成数字

“越快越好”无法指导配置。订单链路可以把目标写成：峰值每秒 20 万条、消息 P99 为 4 KB、端到端 P99 延迟低于 200 ms、允许一个 Broker 故障时继续写入，并在 30 分钟内清空一小时积压。

压测要使用接近生产的消息大小、key 分布、压缩算法、`acks`、副本因子和消费逻辑。每组测试先预热，再记录吞吐、P50/P95/P99 延迟、错误率、CPU、磁盘、网络、GC 和 ISR。一次只改变一个变量，重复运行并保留结果。

```
# Producer 基准：100 万条、每条 1 KiB、不限发送速率
bin/kafka-producer-perf-test.sh \
  --topic perf-orders --num-records 1000000 \
  --record-size 1024 --throughput -1 \
  --bootstrap-server "$BOOTSTRAP" \
  --command-property acks=all compression.type=lz4

# Consumer 基准
bin/kafka-consumer-perf-test.sh \
  --bootstrap-server "$BOOTSTRAP" --topic perf-orders \
  --num-records 1000000 --show-detailed-stats
```

性能脚本适合测量客户端到集群的整体表现，JMH 适合比较某段源码的局部开销。微基准结果需要回到真实集群验证，因为复制、磁盘、网络、GC 和并发请求都会改变最终表现。

19.2 Producer：让每次请求携带更多有效数据

低吞吐且请求很碎时，先观察平均 batch 大小和 records-per-request。适当增加 `batch.size` 与 `linger.ms`，可以让更多记录共享一次压缩、网络往返和 Broker 追加。代价是低流量记录会多等待一小段时间，并占用更多缓冲内存。

`lz4` 和 `zstd` 常用于减少网络与磁盘字节，实际收益取决于消息重复度和 CPU 余量。`buffer.memory` 需要容纳峰值期间的在途批次；buffer pool wait 上升时，应同时检查 Broker 响应速度和网络，而非只扩大缓冲区。

现象	优先检查	调整方向与代价
吞吐低、batch 很小	batch size、linger、key 分布	扩大批次；增加少量排队延迟和内存
网络接近上限	compression ratio、record size	启用或更换压缩；增加客户端与 Broker CPU
send 经常阻塞	buffer pool wait、request latency	先定位下游变慢原因，再评估 buffer.memory
单分区延迟高	热点 key、目标 leader	改善 key 分布或拆分热点；顺序边界会随之变化

19.3 Broker：先判断瓶颈属于磁盘、网络还是线程

Broker 擅长顺序追加和连续读取。日志盘需要稳定的吞吐与延迟，操作系统页缓存需要充足内存，JVM 堆则承担请求对象和运行状态。把大部分内存留给页缓存，通常比盲目扩大堆更有利于日志读写。

网络线程空闲率低时，检查连接数、TLS 开销、响应字节和大量小请求。RequestHandler 空闲率低时，按 API 类型查看请求处理时间和配额。磁盘繁忙时，区分业务写入、副本追赶、日志压缩、分区迁移和历史回放，避免多种重任务同时争用设备。

增加网络线程、请求线程或 I/O 线程只对对应队列积压有效。线程过多会增加调度和内存开销。每次调整都要同时观察队列时间、CPU、磁盘延迟和端到端 P99。

19.4 网络路径：把压缩、零拷贝与 TLS 放在一起测试

Consumer 读取历史数据时，Broker 可能连续发送大量日志页。明文连接下，`FileChannel.transferTo` 有机会利用内核零拷贝路径，CPU 主要处理请求、协议和页引用。开启 SSL 后，字节要经过 `SSLEngine` 加密，CPU 与内存带宽会承担更多工作。

消息压缩能减少磁盘和网络字节。Producer 形成的压缩 batch 可以原样写入日志，并沿 Fetch 路径交给 Consumer。较高的压缩率常常能抵消一部分 TLS 网络成本，同时会增加 Producer 压缩和 Consumer 解压的 CPU。基准测试应固定消息样本，分别测量明文、SSL、不同压缩算法和不同 batch 大小。

观察结果	含义	下一步
SSL 开启后 Broker CPU 明显上升	加密成为主要成本	检查 TLS 算法、连接复用、批量大小和 CPU 余量
网络下降，Producer CPU 上升	压缩把网络成本换成客户端计算	比较端到端 P99 与总资源成本
历史回放时磁盘低、网络高	热数据命中 Page Cache，出口带宽接近上限	增加网络容量、压缩率或分散 leader
历史回放时磁盘延迟升高	工作集超过缓存或后台任务争用磁盘	检查缓存命中、日志迁移、清理和副本追赶

19.5 Consumer：批量拉取与业务处理要匹配

`fetch.min.bytes` 较大时，Broker 会积累更多数据再响应，适合吞吐优先的批量消费；`fetch.max.wait.ms` 给这段等待设置上限。`fetch.max.bytes` 和 `max.partition.fetch.bytes` 控制常规 Fetch 大小。首个非空分区遇到更大的 record batch 时，Consumer 仍会取回该批次以继续推进。

`max.poll.records` 决定一次交给业务多少条记录。批量过大可能让处理时间接近 `max.poll.interval.ms`，引发再均衡；批量过小则增加 poll、提交和下游调用次数。最佳值来自单批处理时延和目标并行度的实测。

同组消费能力最终受分区数限制。增加消费者前先看空闲成员、分区级 lag 和热点分布。业务处理成为瓶颈时，可以按分区建立工作队列，但提交位置必须保持每个分区连续完成。

19.6 分区、副本与可靠性会进入性能预算

分区增加后，写入和消费可以并行扩展，同时会增加文件、索引、复制请求、选主和迁移成本。分区规划应以单分区基准为起点，并验证 Broker 故障后剩余节点能否承接 leader 和流量。

`acks=all`、min ISR 和副本因子决定一次成功写入要等待多少复制工作。可靠性目标固定以后，再通过批量、压缩、硬件和 leader 分布提高性能。缩小确认条件会改变数据承诺，应作为可靠性决策单独评审。

19.7 用指标把症状定位到具体环节

症状	关键证据	常见方向
Producer P99 上升	batch、buffer wait、request queue/local/remote time	区分成批等待、Broker 排队、磁盘和副本确认
Broker CPU 高	API 请求数、request percentage、压缩/TLS、GC	合并小请求、优化压缩、配额隔离、检查堆
磁盘延迟高	日志目录、清理、迁移、副本追赶、页缓存命中	错峰后台任务、增加设备吞吐、重新分布分区
Consumer lag 增长	分区 lag、消费速率、处理时延、再均衡	处理能力、热点分区、Fetch 批量或组稳定性
吞吐达标但延迟抖动	P99/P99.9、GC、磁盘尾延迟、ISR 变化	关注尾部而非平均值，保留故障与突发余量

19.8 推荐的调优顺序

1. 固定消息、流量、可靠性配置和硬件，建立可重复基线。
2. 确认 key 与分区分布，排除热点和 leader 倾斜。
3. 调 Producer 批量与压缩，观察吞吐、P99、CPU 和网络。
4. 调 Consumer Fetch 与业务批量，观察处理时延和再均衡。
5. 根据 Broker 队列、磁盘和网络证据调整线程或硬件。
6. 加入 Broker 故障、积压回放和分区迁移，再验证 SLO。

工具与源码锚点

- `tools/.../ProducerPerformance.java`、`ConsumerPerformance.java`：端到端吞吐基准。
- `jmh-benchmarks/`：客户端、协议、存储和协调器的微基准。
- `trogdor/`：多节点负载与故障场景测试框架。
- `RecordAccumulator.java`、`Sender.java`：Producer 成批、排队与发送。
- `FileRecords.java`、`PlaintextTransportLayer.java`、`SslTransportLayer.java`：明文零拷贝与 SSL 发送路径。
- `server/.../quota/ClientQuotaManager.java`：共享集群的配额与节流。

监控、运维与滚动变更

进程存活只说明服务还在运行。可靠监控还要回答：元数据能否更新、分区能否读写、资源是否充足，以及业务事件能否按时处理。

20.1 四层信号

层次	关键问题	代表信号
控制面	元数据能否形成多数派并及时传播？	active controller、quorum leader/epoch、voter lag
数据面	分区是否可写、可读、复制健康？	offline、under-replicated、under-min-ISR、ISR 变化
资源面	节点是否逼近物理极限？	磁盘容量/延迟、网络、CPU、GC、请求线程空闲率
业务面	事件是否在允许时间内到达并完成？	produce error/latency、consumer lag 时间、处理成功率

20.2 Lag 必须配业务时间

Lag 记录数相同，业务影响可能完全不同。积压 100 万条、每秒净消化 50 万条时，两秒即可追平；积压 1 万条、每秒只能处理 10 条时，则需要近 17 分钟。

因此告警应同时观察最老未处理事件的年龄、生产速率、消费速率和预计清空时间。业务 SLO 通常更关心事件延迟，而非一个孤立的记录数。

20.3 五分钟体检命令

```
bin/kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \
  --describe --unavailable-partitions
bin/kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \
  --describe --under-replicated-partitions
bin/kafka-topics.sh --bootstrap-server "$BOOTSTRAP" \
  --describe --under-min-isr-partitions --exclude-internal
bin/kafka-metadata-quorum.sh --bootstrap-server "$BOOTSTRAP" \
  describe --status
bin/kafka-log-dirs.sh --bootstrap-server "$BOOTSTRAP" --describe
bin/kafka-consumer-groups.sh --bootstrap-server "$BOOTSTRAP" \
  --describe --all-groups
```

20.4 滚动升级的安全顺序

1. 核对源/目标版本支持矩阵、弃用项、客户端兼容与回滚边界。
2. 备份并审查配置，验证监控、告警、容量和故障演练。
3. 先升级 Controller/Broker 二进制，按容错预算一次处理有限节点。
4. 每一步等待 ISR、分区可用性、请求错误和业务 SLO 恢复稳定。
5. 全部节点稳定后，再按目标版本说明升级 metadata feature。

Feature level 的提升可能改变持久化格式或集群行为，回退边界通常比二进制滚动更严格。把“所有节点已运行新二进制”和“启用新 feature”分成两个可独立验证的阶段，可以保留清晰的回滚窗口。

20.5 分区迁移

分区迁移会同时占用源端和目标端的磁盘、网络与复制线程。执行前先生成计划并核对目标 Broker，执行时按集群余量限速，同时观察 ISR、磁盘空间、请求延迟和业务 lag。

命令结束后，还要确认副本已经到达目标位置、leader 分布合理、ISR 恢复完整，并且旧日志目录已经释放。

20.6 Admin API：异步且元数据最终传播

`Admin` 方法通常返回结果对象和 Future。Controller 接受变更并更新权威状态后，Future 才会完成。新元数据传播到所有 Broker 还需要一点时间，因此紧接着向另一个 Broker 查询时，可能暂时看不到刚创建的资源。

自动化脚本应等待 Future，逐个检查实体结果，再围绕明确的状态条件做有界重试。创建、删除、扩分区、改配置、leader election 和 reassignment 还需要 dry-run、精确目标、超时、审计记录和执行后验证。

故障排查：从症状到证据链

排障从影响范围和时间线开始，再判断问题位于数据面还是控制面。每一步动作都要由证据支持，并用同一组指标验证结果。

21.1 客户端连不上

1. 确认失败发生在 DNS、TCP、TLS、SASL、授权还是 Kafka 协议阶段。
2. 从客户端所在网络测试 bootstrap 地址，再检查元数据返回的 advertised 地址。
3. 对 TLS 检查 SAN、证书链、时钟与信任库；对 SASL 检查机制和 principal。
4. 只在证据指向 Broker 时再查 listener、连接配额与网络线程。

21.2 Producer 超时或吞吐骤降

- 比较 buffer pool 等待、batch size、request latency、record error/retry。
- 确认是所有分区还是单个热点分区/leader。
- 检查 ISR、min ISR、配额、Broker 请求队列、磁盘与网络。
- 区分 `TimeoutException` 是元数据等待、缓冲等待还是 delivery timeout。

21.3 Lag 持续增长



先把 lag 展开到分区维度。所有分区一起增长，通常说明整体处理能力低于输入；只有一个分区增长，重点检查热点 key、该分区 leader 的磁盘与网络，以及对应任务是否卡住。

如果组成员频繁变化，再查看 poll 间隔、session timeout、实例重启和网络抖动。频繁再均衡会造成停顿，并让未提交记录反复处理。

21.4 重复与疑似丢失

症状	优先验证
Kafka 内出现重复 record	Producer 幂等是否启用、是否换了身份、上游是否重复调用
业务副作用重复	处理完成与 offset 提交之间是否崩溃、下游是否按 event ID 幂等
消费者“少读”	是否提交过头、过滤/反序列化失败、read_committed、offset 越界
Broker 确认后数据消失	acks/min ISR、unclean election、日志截断、错误集群/Topic

21.5 Under-replicated / Under-min-ISR

先按 Broker 和日志目录聚合异常分区，寻找共同故障点。随后检查 follower lag、网络、磁盘空间、长 GC 和正在执行的副本迁移。

基础设施恢复后，继续观察 ISR 是否回到目标副本数。业务选择紧急降级时，应记录风险接受者、受影响 Topic、有效时间窗和恢复阈值。

21.6 KRaft 无法变更元数据

先确认 controller quorum 是否有 leader 和多数派，再检查 voter endpoint 连通性、node ID、cluster ID，以及日志目录与 `meta.properties` 是否匹配。

重新 format 会改写目录身份，只适用于明确的空目录初始化或受控恢复流程。控制面失效后，Broker 可能继续使用已有元数据处理部分数据请求，这段短暂可用期仍然需要按控制面故障处理。

> **排障输出模板：**影响范围；开始时间；最后已知正常；关键错误原文；控制面状态；分区/ISR；客户端指标；资源指标；已做动作与结果；下一步及回滚条件。

源码精读实验与学习路线

每个实验只回答一个问题，并沿调用链找到状态、成功条件和恢复入口。这样读源码更容易形成完整结论。

实验 A：跟踪一次 Producer 发送

1. 在 `KafkaProducer.doSend` 找到序列化、分区与 accumulator append。
2. 在 `RecordAccumulator` 观察 batch 创建、ready 与 drain 条件。
3. 在 `Sender.sendProducerData` 找请求、超时、in-flight 与响应回调。
4. 回答：Future 何时完成？什么错误会重试？总时限由谁裁决？

实验 B：跟踪 Broker 追加

1. 从 `KafkaApis.handleProduceRequest` 进入 `ReplicaManager.appendRecords`。
2. 进入分区 leader 追加与 `UnifiedLog.appendAsLeader`。
3. 找到 `acks=all` 下延迟完成条件。
4. 回答：本地追加成功与客户端成功之间还有哪些条件？

实验 C：制造再均衡

1. 创建 6 分区 Topic，启动 2 个同组消费者并记录分配。
2. 启动第 3 个消费者，观察 revoke/assign、generation/epoch 和停顿。
3. 让一个消费者处理超过 `max.poll.interval.ms`，再观察成员变化。
4. 切换 cooperative assignor，对比撤销范围。

实验 D：观察 HW 与事务可见性

1. 在多 Broker 环境写入数据并观察副本 LEO/HW。
2. 暂停 follower 或限速，观察 ISR 与 all-acks 写入。
3. 开启事务，写一批不提交的数据，对比 `read_uncommitted` 与 `read_committed`。

实验 E：验证保留与压缩

1. 创建短 segment、短 retention 的实验 Topic，观察按 segment 删除的延迟。
2. 创建 compact Topic，反复写同 key 与 tombstone。
3. 等待 cleaner 后用 dump log 工具检查 offset 没有重编号、旧值逐步消失。

推荐阅读顺序



> **完成标准：**能画出调用链、说清状态存在哪里、定义成功条件、列出故障恢复入口，并能用一个定向测试验证关键不变量。

术语、配置与参考书目

23.1 高频术语

术语	简明定义
LEO	某副本日志末端，等于下一条追加位置。
HW	普通消费者可稳定读取的复制边界。
LSO	事务隔离下的最后稳定 offset，受未完成事务影响。
ISR	当前与 leader 保持足够同步的副本集合。
Lag	日志末端与消费位置之差；应结合事件年龄解释。
Epoch	区分领导任期、生产者身份代际或成员代际的单调版本。
Fencing	拒绝旧身份继续写入，防止双主或僵尸实例。

23.2 配置组合速查

目标	起始思路	仍需验证
关键生产可靠性	RF=3、min ISR=2、acks=all、幂等	跨故障域、副本恢复、业务幂等
生产吞吐	合适 batch/linger、压缩、足够 buffer	P99 延迟、CPU、消息分布
消费稳定	批量与处理时间匹配、显式提交边界	外部依赖、再均衡、分区热点
状态 Topic	compact，合理 tombstone 保留	全量恢复、delete 组合风险
安全	TLS + 身份认证 + ACL + 网络隔离	轮换、审计、插件与管理面

23.3 参考书目

- Gwen Shapira, Todd Palino, Rajini Sivaram, Neha Narkhede, *Kafka: The Definitive Guide, 2nd Edition*。
- Nishant Garg, *Apache Kafka*, Packt, 2013。
- Nishant Garg, *Learning Apache Kafka, 2nd Edition*。
- Dylan Scott, *Kafka in Action*。
- Emil Koutanov, *Effective Kafka*。
- Bill Bejeck, *Kafka Streams in Action*。
- Mitch Seymour, *Mastering Kafka Streams and ksqlDB*。
- Ben Stopford, *Designing Event-Driven Systems*。

以上资料覆盖多个 Kafka 版本。命令、默认值、KRaft、组协议和安全配置应按目标发行版核对。

23.4 一手资料优先级

1. 目标发行版官方文档、Javadoc、配置文档和发布说明。
2. Kafka 源码、对应测试、生成协议 schema。
3. 已通过的 KIP，用于理解设计动机与兼容边界。

4. 书籍、演讲与博客，用于解释和案例，但需回到一手资料核验。

23.5 上线前最终检查

- [ ] 顺序键、分区数和未来扩分区策略已明确。
- [ ] 副本因子、min ISR、acks 与故障域形成一致承诺。
- [ ] Producer/Consumer 的成功、失败、重试和超时都有指标。
- [ ] 业务副作用有稳定 event ID、幂等或事务协调。
- [ ] 容量覆盖保留、复制、回放、迁移和清理的非稳态峰值。
- [ ] 安全覆盖 TLS、认证、授权、网络、密钥轮换与管理面。
- [ ] 告警连接到可执行 Runbook，恢复流程经过演练。
- [ ] 升级方案明确二进制阶段、feature 阶段和不可逆边界。

结语：用四个问题驾驭 Kafka

面对一个新的 Kafka 设计，先问四个问题：数据落在哪个分区？进度由谁记录？写入在什么条件下算成功？故障后从哪里恢复？顺序、吞吐、重复、积压和可用性，都可以沿着这四个问题继续分析。